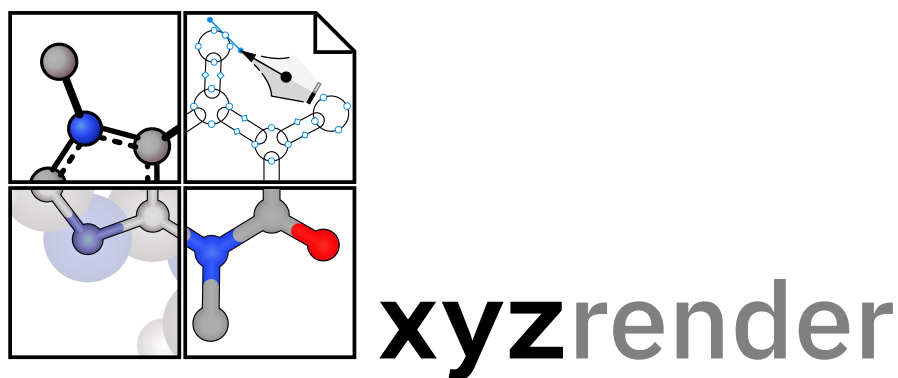




API Manual - Typst port

Every `xyzrender(...)` keyword, explained and shown.

Typst port of xyzrender by Alister S. Goodfellow (github.com/aligfellow/xyzrender) — see Section 14.

1 Why this exists

`xyzrender` started as a Python library - read a structure, perceive its bonds, draw it. That's a workflow most chemistry writing still routes through a separate tool: render a figure somewhere else, export a PNG, paste it in. This port exists to close that gap for Typst specifically: `xyzrender` sits directly in your source, next to the prose that references it, and recompiles the instant the structure or a keyword changes. No exported image to go stale, no second tool to keep open.

Making that work meant moving the actual chemistry — bond perception, aromaticity detection, all of it — into a small Rust library compiled to WebAssembly, since Typst has no Python underneath it to lean on. Everything **around** that core (styling, presets, SVG emission) is plain Typst, and it mirrors the upstream Python module layout file-for-file on purpose: if upstream adds a preset or fixes a rendering edge case, translating the change over means finding the one matching file, not hunting through a reorganized port. Where this manual calls out a place we deliberately **don't** match upstream — a different default, a Typst-only convenience — you'll see it flagged in a box like this:

DEVIATES FROM UPSTREAM

Called out whenever this port's behaviour differs from upstream on purpose.

This manual is organized around **why** you'd reach for a feature, not just what it's called. Each section opens with what the feature is for, then the keyword table, then a live example rendered from the same caffeine molecule throughout — so every comparison in this document is apples to apples, and you can tell at a glance which keyword produced which visual change.

Contents

1	Why this exists	1
2	Call signature	3
2.1	Document-wide defaults	3
2.2	PDB input	4
3	Built-in presets	5
3.1	Aromatic vs. Kekulé	6
4	Atom geometry	7
5	Bonds	8
5.1	Manual bond rules	8
6	TS / NCI overlays	9
7	Shading / fog	10
8	Hydrogens	11
9	VDW overlay	12
10	Labels	13
11	Style regions	14
11.1	Selector tokens	14
12	Canvas & framing	16
13	Out of scope (for now)	17
14	Citation	18

2 Call signature

Every call to `xyzrender` follows the same shape: a molecule (the only required argument), a handful of positional conveniences that mirror `image(...)`, and then an open-ended set of style overrides.

```
xyzrender(source, config, width, height, rotate, frame, background, hy,
no_hy, ..overrides)
```

Param	Default	Notes
<code>source</code>	—	XYZ or PDB bytes from <code>read("file", encoding: none)</code> , an inline XYZ/PDB string, or a bare path string once <code>reader</code> is bound — format is auto-detected (Section 2.2)
<code>config</code>	"default"	Built-in preset name or path to a <code>.json</code> config file
<code>width</code>	auto	Passed to <code>image()</code>
<code>height</code>	auto	Passed to <code>image()</code>
<code>rotate</code>	(:)	Rotation of the molecule
<code>frame</code>	0	Which frame to pick from a multi-frame XYZ (ignored for PDB)
<code>background</code>	none	<code>none</code> = transparent; otherwise CSS-style hex or named colour
<code>hy</code>	none	<code>none</code> = auto-hide C–H; <code>true</code> = show all; <code>(1, 3) / "1,3-5"</code> = show only those H
<code>no_hy</code>	false	<code>true</code> = hide every hydrogen

2.1 Document-wide defaults

Rendering the same molecule several times with the same non-default style — say, every figure in a paper uses the `tube` preset at a fixed width — shouldn't mean repeating those settings at every call site. Typst already has a tool for “bind some arguments once, reuse the rest”: `.with()`. Rebind the imported name and every later call to it carries the new defaults automatically, while a kwarg passed at the call site still overrides them:

```
#import "@preview/xyzrender:0.1.0": xyzrender

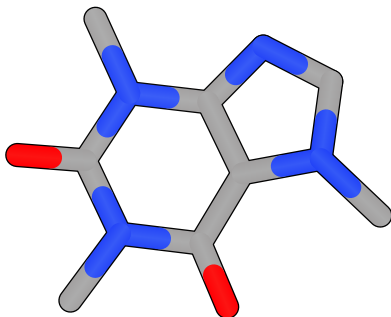
#let xyzrender = xyzrender.with(reader: p => read(p, encoding: none))
#let xyzrender = xyzrender.with(config: "tube", canvas_size: 150, fog: false)

// Every following call gets config/canvas_size/fog merged underneath its own
// kwargs.
#xyzrender("examples/caffeine.xyz")
#xyzrender("examples/caffeine.xyz", rotate: (y: 90deg))
// config: kwarg here still wins
#xyzrender("examples/caffeine.xyz", config: "wire")
```

This is plain Typst — no state, no show rule, nothing package-specific — so it's the recommended way to set defaults for a whole document or a whole section.

2.2 PDB input

Alongside XYZ, `xyzrender` reads PDB files — `ATOM/HETATM` coordinates, `CONNECT` connectivity, and the `CRYST1` unit cell. Format is auto-detected from content, so the same call works for either:



```
#xyzrender(read("/examples/caffeine.pdb"), config: "tube") — auto-detected as PDB, no format:  
                                argument needed
```

What actually happens with the bonds depends on what the file gives you, mirroring upstream exactly:

File contains	What you get
<code>CONNECT</code> records	Bonds straight from the file — no geometric detection, no aromaticity. Order is always single; <code>CONNECT</code> carries no bond-order information to perceive from.
No <code>CONNECT</code> , has <code>CRYST1</code>	Geometric bond detection runs, but the aromaticity/bond-order optimizer is skipped — a periodic cell is a strong enough hint that this isn't a lone gas-phase molecule to treat cautiously.
Neither	Full pipeline, identical to an XYZ file with the same coordinates — bond orders and aromaticity fully perceived.

NOTE

PDB support here is deliberately narrow, matching what upstream's own parser extracts: coordinates, elements, connectivity, and the unit cell; nothing else.

`source` normally still needs a `read(...)` call at the call site, for a reason that's easy to miss the first time you hit it: a **package's** own `read()` resolves paths relative to the package, not your document, so `xyzrender` can't just accept `"mol.pdb"` and go find it in your project. Binding a `reader` once gets you the bare-path syntax anyway:

```
#let xyzrender = xyzrender.with(reader: p => read(p, encoding: none))  
#xyzrender("Structures/caffeine.pdb")
```

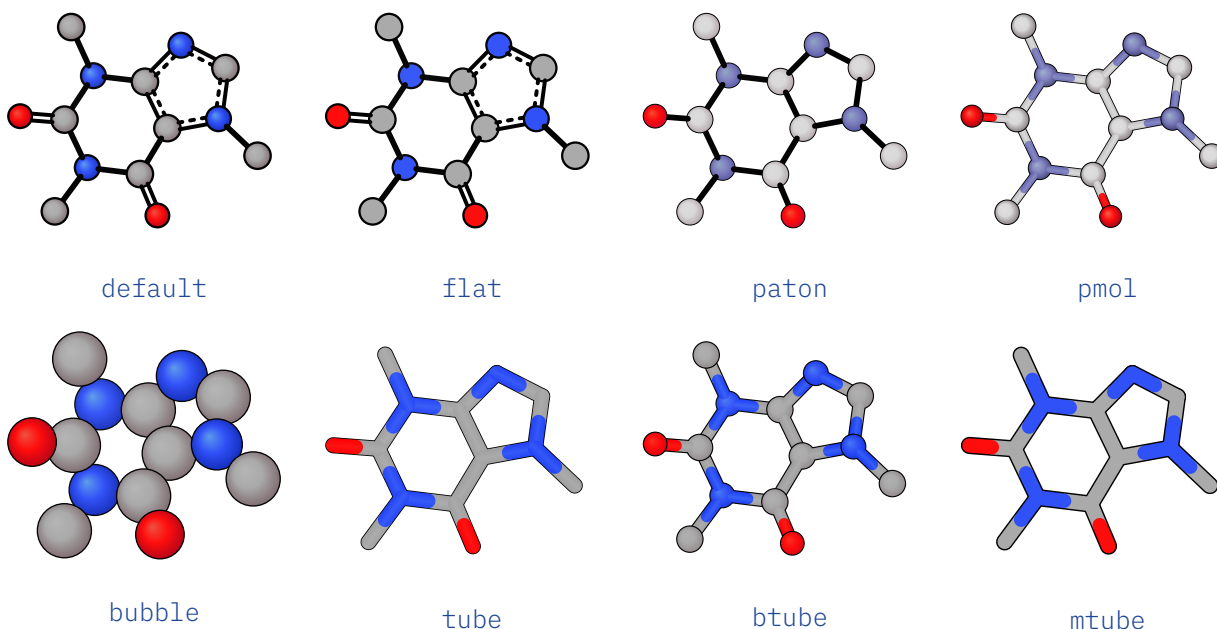
This works because `read(p, encoding: none)` is written in **your** document — even though it only actually runs once `xyzrender` calls `reader(...)` from deep inside the package, Typst resolves the path against wherever that code was written, not wherever it ends up being called from. It's the same trick that lets a content block you hand to someone else's package keep using your own relative `image()` paths.

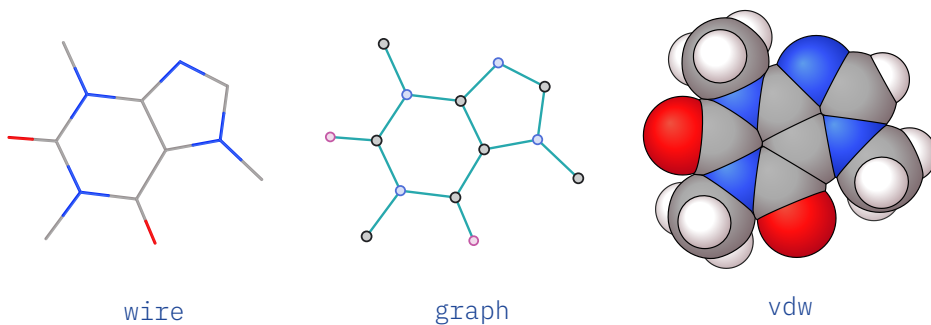
3 Built-in presets

A preset is a named starting point (a `.json` file layered on top of `default.json`) that only needs to specify what it changes. Pass one by name via `config: "tube"`, or point at your own file (`config: "my-style.json"`) to layer custom defaults the same way. Eleven presets ship with the package, spanning the usual range from ball-and-stick to space-filling to line-drawing:

Name	Description
<code>default</code>	Ball-and-stick, gradient atoms, fog, dark bond outlines
<code>flat</code>	No gradient shading; flat element fills
<code>paton</code>	Subtle fog, soft palette
<code>vdw</code>	Space-filling: huge atoms, <code>atom_interlocking: true</code> , <code>hide_bonds: true</code>
<code>tube</code>	Thin atoms, thick coloured bonds
<code>btube</code>	Bigger atoms + cylinder shading on bonds
<code>mtube</code>	Middle ground between tube and btube
<code>bubble</code>	Oversized atom discs, no bonds
<code>graph</code>	Diagram-style: <code>atom_wash: 0.78</code> , flat fills
<code>pmol</code>	Skeletal-leaning publication look
<code>wire</code>	<code>atom_scale: 0</code> — bonds only, with dot fallback on isolated atoms

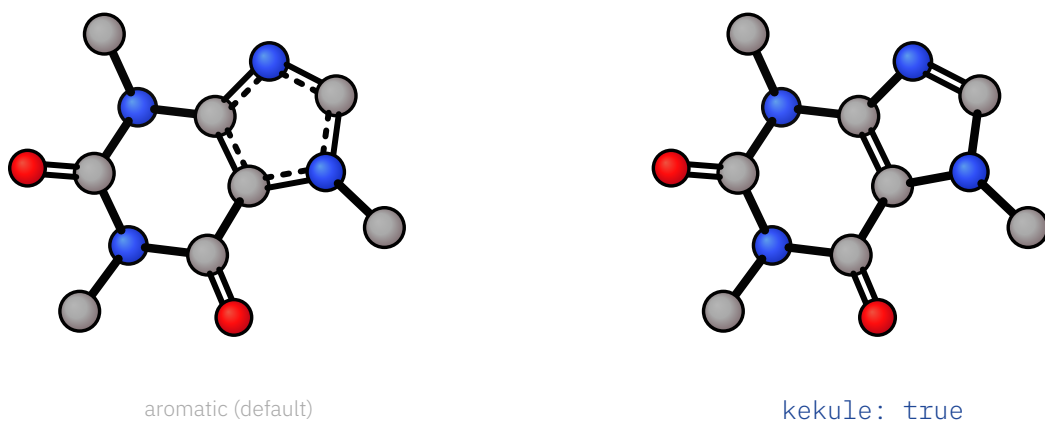
Same caffeine molecule, every preset, so you can compare silhouettes directly:





3.1 Aromatic vs. Kekulé

xyzgraph perceives aromaticity directly from geometry, so aromatic rings render by default as a solid outer ring with a dashed inner ring — no input annotation required. If you'd rather see textbook alternating single/double bonds instead, `kekule: true` switches the whole molecule over:

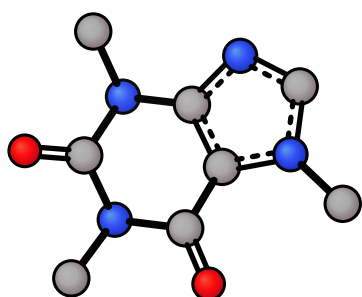


4 Atom geometry

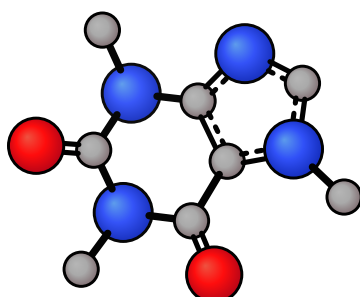
These keys control how each atom sphere is sized, outlined, and filled. Sizes are relative — `atom_scale` and `vdw_scale` are multipliers on the underlying per-element radius table, so scaling one preset up or down never requires touching individual atoms. Setting `atom_scale: 0` turns off atom discs entirely, which is how the `wire` preset gets its bonds-only look.

Key	Default	Effect
<code>atom_scale</code>	1.0	Global sphere size multiplier
<code>vdw_scale</code>	1.0	Multiplier for the VDW overlay radii
<code>h_scale</code>	0.6	Shrink factor applied to H radii for primary atoms
<code>vdw_h_scale</code>	0.7	Shrink factor applied to H in the VDW overlay
<code>atom_stroke_width</code>	1.5	Per-atom stroke width (px @ ref scale)
<code>atom_stroke_color</code>	"#000000"	Per-atom stroke colour; "atom" = use element colour
<code>atom_wash</code>	0.0	Blend fill toward white (flat-fill only); 0.78 ≈ graph-style
<code>radius_scale</code>	(:)	Per-element multipliers, e.g. ("H": 1.2)
<code>colors</code>	(:)	Per-element fill overrides, e.g. ("N": "olive")
<code>atom_interlocking</code>	false	Clip overlapping primary spheres into CineMol-style silhouette polygons
<code>vdw_interlocking</code>	false	Same clipping applied to the VDW overlay layer
<code>vdw_interlock_samples</code>	64	Perimeter samples per sphere on the silhouette great circle

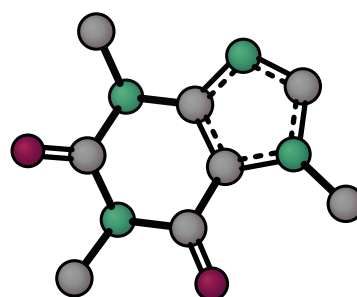
`radius_scale` and `colors` are the two you'll reach for most: both take a dict keyed by element symbol, so you only override what you need to and everything else keeps the preset's values.



default



`radius_scale: (N: 1.8, O: 1.8)`



`colors: (N: olive, O: maroon)`

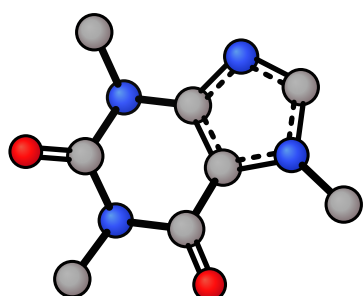
5 Bonds

Bond order and connectivity come from geometry (via xyzgraph), so most molecules need no manual bond input at all. These keys are for styling what's already perceived, or for the occasional correction (`bond` / `unbond`) when perception gets an edge case wrong. In practice that's a rare need: geometric bond perception gets ordinary organic structures right often enough that `bond/unbond` are the two keys in this whole reference you're least likely to ever type.

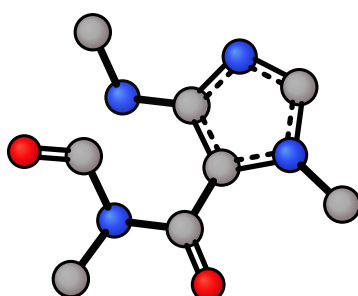
Key	Default	Effect
<code>bond_width</code>	5.0	Bond pill thickness (px @ ref scale)
<code>bond_color</code>	"#333333"	Flat bond fill colour
<code>bond_color_by_element</code>	false	Half-and-half element-coloured bonds
<code>bond_outline_color</code>	"#000000"	Dark outline colour
<code>bond_outline_width</code>	0.0	Dark outline width (0 = no outline)
<code>bond_gradient</code>	false	Apply cylinder shading along each bond
<code>bond_gradient_strength</code>	1.0	How dark the cylinder rim goes
<code>bond_gap</code>	0.6	Multi-bond spacing as a fraction of <code>bond_width</code>
<code>bond_orders</code>	true	When false, every bond renders as a single
<code>kekule</code>	false	When true, aromatic rings keep alternating 1/2 instead of the dashed-pair
<code>hide_bonds</code>	false	Suppress bond emission entirely (used by <code>vdw</code> , <code>bubble</code>)
<code>bond</code>	()	Force-add bonds, 1-indexed pair specs like ("2-3", "4-5")
<code>unbond</code>	()	Drop perceived bonds, 1-indexed pair specs like ("1-2")

5.1 Manual bond rules

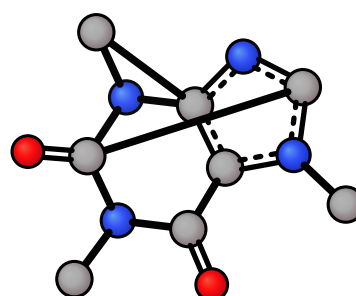
`bond` and `unbond` apply **before** everything else in the render pipeline — including hydrogen auto-hiding — so unbonding an atom's only heavy neighbour will correctly leave it visible as a free H rather than vanishing.



baseline



unbond: ("6-9")

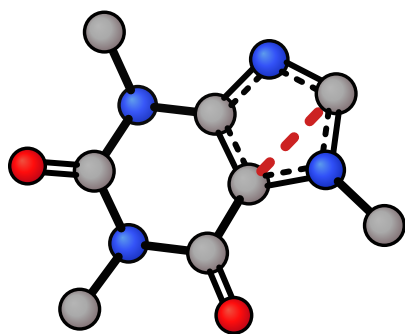


bond: ("3-4", "5-6")

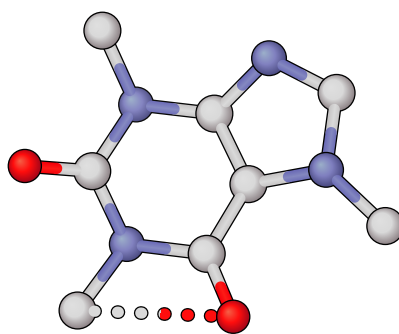
6 TS / NCI overlays

Two dashed-line overlays for the two things covalent bonds can't represent: `ts_bonds` for forming/breaking bonds in a transition-state structure, `nci_bonds` for non-covalent contacts (H-bonds, π -stacking, ...). Both take pairs of atom indices (a pair that already exists as a covalent bond gets restyled in place, a pair that doesn't gets added as a new edge). Index pairs are subject to `index_base` (1-indexed by default; see Section 9).

Key	Default	Effect
<code>ts_bonds</code>	<code>()</code>	Pairs like <code>((1, 5),)</code> – dashed stroke
<code>ts_color</code>	<code>none</code>	<code>none</code> falls back to <code>bond_color</code>
<code>ts_element</code>	<code>false</code>	Atom-coloured dash halves (needs <code>bond_color_by_element: true</code> and <code>ts_color: none</code>)
<code>ts_dash</code>	<code>(1.2, 2.2)</code>	(length, gap) as <code>bond_width</code> multipliers; tube presets override to <code>(0.5, 3)</code>
<code>ts_width</code>	<code>1.2</code>	Stroke width as a <code>bond_width</code> multiplier
<code>nci_bonds</code>	<code>()</code>	Pairs like <code>((1, 8),)</code> – dotted stroke
<code>nci_color</code>	<code>none</code>	As <code>ts_color</code>
<code>nci_element</code>	<code>false</code>	As <code>ts_element</code>
<code>nci_dash</code>	<code>(0.08, 2.0)</code>	Dotted pattern
<code>nci_width</code>	<code>1.0</code>	Stroke width multiplier



`ts_bonds: ("1-5"), ts_color: "#cc2222"`

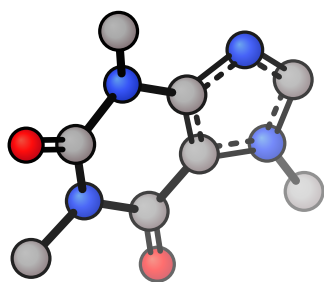


`nci_bonds: ("8-14"), nci_element: true`

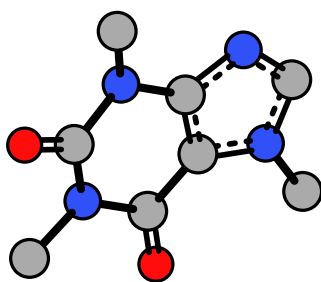
7 Shading / fog

Two independent effects give atoms a sense of depth without a real 3D renderer: a radial gradient across each atom sphere (pseudo-lighting), and fog (blending farther atoms toward the background colour, like atmospheric haze). Both can be tuned or turned off per call.

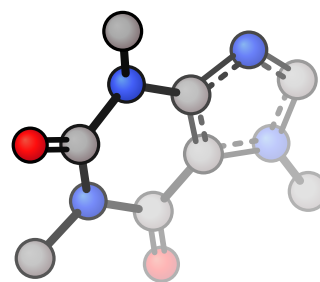
Key	Default	Effect
<code>gradient</code>	<code>true</code>	Radial sphere gradient on atom fills
<code>atom_gradient_strength</code>	<code>1.0</code>	Overall lighten/darken intensity
<code>hue_shift_factor</code>	<code>0.2</code>	Gradient hue shift toward 240°/60°
<code>light_shift_factor</code>	<code>0.2</code>	Gradient lightness shift
<code>saturation_shift_factor</code>	<code>0.2</code>	Gradient saturation shift
<code>fog</code>	<code>true</code>	Depth-blend atoms toward the background
<code>fog_strength</code>	<code>1.2</code>	How aggressively fog blends with depth
<code>fog_color</code>	<code>none</code>	Fog target colour; <code>none</code> = derive from <code>background</code>



default (gradient + fog on)



gradient: false, fog: false



fog_strength: 3.0

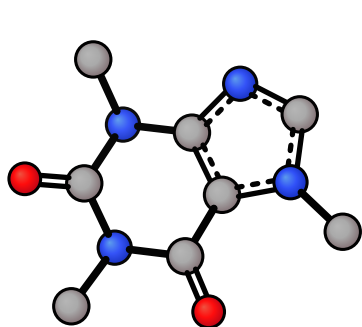
8 Hydrogens

Structures loaded straight from geometry optimisers tend to carry every hydrogen, which clutters a rendered figure fast. The three flags below give fine control, but in practice you'll usually want the friendlier `hy` / `no_hy` call-site kwargs from Section 2, which just set these three for you.

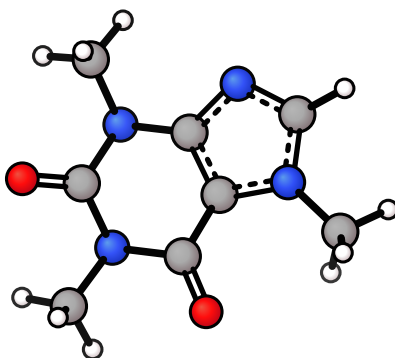
Key	Default	Effect
<code>hide_h</code>	<code>false</code>	Hide every H except <code>show_h_indices</code>
<code>show_h_indices</code>	<code>()</code>	H atoms kept visible when <code>hide_h</code> is on
<code>auto_hide_h</code>	<code>true</code>	When <code>hy</code> isn't passed, hide C-H hydrogens automatically

TIP

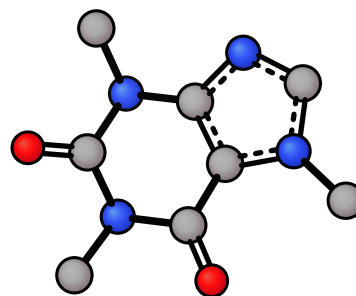
Prefer `hy` / `no_hy` over the three keys above directly: `hy: none` (auto-hide C-H, the default), `hy: true` (show all), `hy: (15, 16) / hy: "15-16"` (show only those H), `no_hy: true` (hide every H).



`hy: none` (default, auto-hide)



`hy: true`



`no_hy: true`

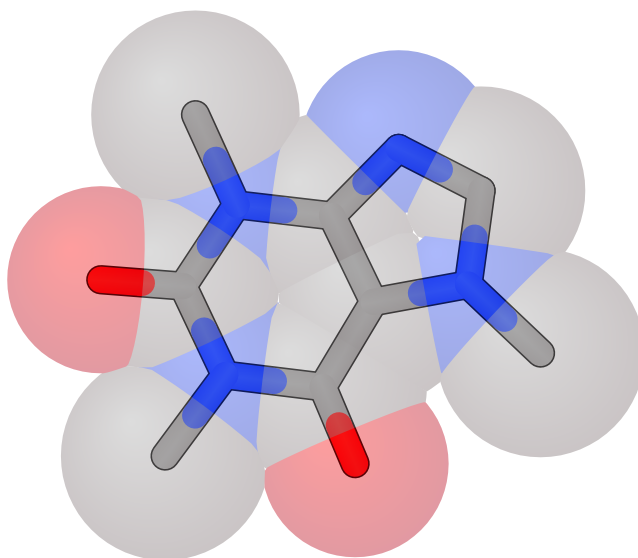
9 VDW overlay

A translucent van der Waals sphere layer drawn on top of the primary atom rendering (useful for showing a pocket, a steric clash, or just the molecular volume alongside a ball-and-stick skeleton). Inactive by default; setting `vdw_indices` to anything other than `none` turns it on.

Key	Default	Effect
<code>vdw_indices</code>	<code>none</code>	<code>()</code> or <code>" "</code> = every atom; <code>(1, 6, 8)</code> = subset; also accepts selector strings like <code>"1,4-7,h"</code>
<code>vdw_opacity</code>	<code>0.5</code>	Group opacity of the overlay layer
<code>vdw_gradient_strength</code>	<code>1.6</code>	Rim darkening for overlay spheres
<code>vdw_outline_width</code>	<code>none</code>	Inherit <code>atom_stroke_width</code> ; <code>0</code> = no outline
<code>vdw_outline_color</code>	<code>none</code>	Inherit <code>atom_stroke_color</code> ; <code>"atom"</code> = element colour
<code>index_base</code>	<code>1</code>	Base for user-supplied indices (<code>vdw_indices</code> , <code>ts_bonds</code> , <code>nci_bonds</code> , <code>hy</code>); set <code>0</code> for raw 0-indexed values

NOTE

`index_base` is document-wide within a single `xyzrender` call — it governs every user-facing index list (`vdw_indices`, `ts_bonds`, `nci_bonds`, `hy`), not just this overlay. It defaults to `1` so plain atom numbers match what you'd read off a molecular viewer.

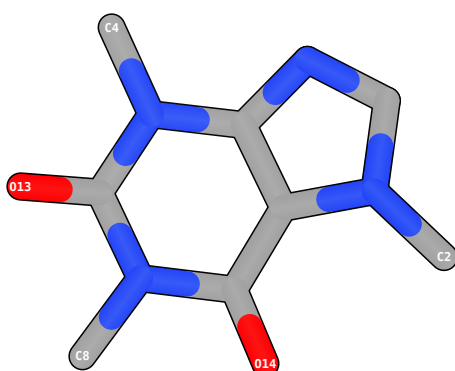


config: "tube", vdw_indices: 0, vdw_opacity: 0.4, vdw_interlocking: true

10 Labels

Per-atom text labels (element symbol, index, or both) drawn at each atom centre. Useful when a figure needs to be cross-referenced from running text (“the highlighted N1...”).

Key	Default	Effect
<code>show_indices</code>	<code>false</code>	Emit a per-atom text label
<code>idx_format</code>	<code>"sn"</code>	<code>"sn" = C1, "s" = C, "n" = 1 (1-indexed)</code>
<code>label_font_size</code>	<code>11.0</code>	px @ ref scale
<code>label_color</code>	<code>"#222222"</code>	Text fill



`show_indices: true, idx_format: "sn", label_color: "white", label_font_size: 24`

11 Style regions

Everything so far applies one setting to the whole molecule. Style regions let you carve out a subset of atoms (a ligand, a metal centre, a functional group) and apply a different config to just that subset, while the rest of the molecule keeps the base style.

Key	Default	Effect
<code>style_regions</code>	<code>()</code>	Explicit list of regions, each <code>(indices: (0,1,2), config: (atom_scale: 4.0))</code> . <code>indices</code> are 0-indexed.
<code>region_specs</code>	<code>none</code>	Selector-keyed shorthand: <code>("het": "flat", "5-12": (atom_wash: 0.6))</code>

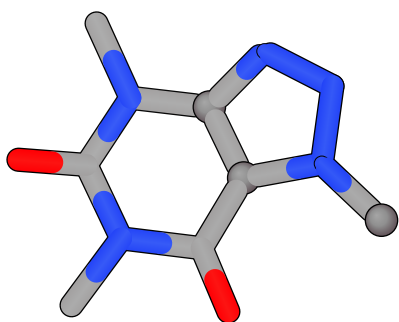
`style_regions` is the explicit form: you supply a list of atom indices and a config dict per region. `region_specs` is the shorthand most people reach for instead: keys are selector tokens (below), values are either a config dict or the name of another preset to apply wholesale. If both are given, `style_regions` claims its atoms first and `region_specs` only fills in whatever's left over.

11.1 Selector tokens

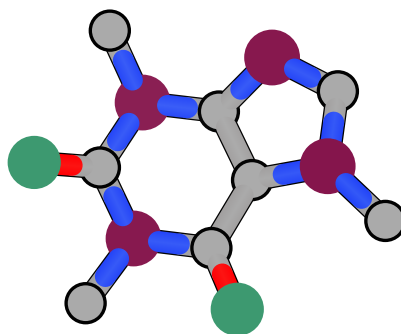
Selector strings show up in several places — `region_specs` keys, `vdw_indices`, `hy`, `ts_bonds/nci_bonds` — and are resolved the same way everywhere, mirroring upstream `xyzrender/selectors.py`:

Token	Matches
<code>M</code>	All metals
<code>sbm</code>	s-block metals
<code>L</code>	Non-metals; when the graph has metals, narrowed to atoms bonded to a metal ("ligand")
<code>het</code>	Non-C, non-H, non-metal; same metal-narrowing as <code>L</code>
<code>hal</code>	Halogens
<code>pnict</code>	Pnictogens
<code>chal</code>	Chalcogens
<code>noble</code>	Noble gases
<code>triel</code>	Group 13
<code>tetrel</code>	Group 14
Element symbol	e.g. <code>N</code> , <code>O</code> — exact match
<code>N</code> (digit)	1-indexed atom number
<code>N-M</code>	1-indexed inclusive range
<code>all</code> , <code>*</code>	Every atom
<code>tok1</code> , <code>tok2</code> , ...	Union of any of the above

Only per-atom fields make sense on a region — `atom_scale`, `atom_stroke_width`, `atom_stroke_color`, `gradient`, `atom_gradient_strength`, `atom_wash`, and per-element `colors` overrides. Global fields (`canvas_size`, `fog`, `background`, etc.) are taken from the base config and ignored if set on a region.



explicit style_regions by 0-indexed atom list



selector-keyed region_specs

12 Canvas & framing

The last group controls the canvas itself: how much space surrounds the molecule, how it's oriented before drawing, and how its scale behaves across different molecules and output sizes.

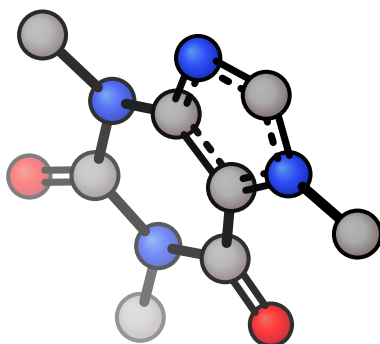
Key	Default	Effect
<code>canvas_size</code>	800	Reference canvas in px (drives bond / font scaling and zoom)
<code>padding</code>	10.0	Inner padding around the molecule (upstream default is 20.0)
<code>background</code>	"#ffffff"	Fill rect at the back; <code>none</code> skips it
<code>tight_fit</code>	<code>true</code>	Fit the canvas to per-atom extents so only <code>padding</code> surrounds the ink; <code>false</code> = upstream-style fit (pads every side by the largest atom radius)
<code>auto_orient</code>	<code>true</code>	PCA-orient the molecule (longest axis → x) before any <code>rotate</code>
<code>fixed_span</code>	<code>none</code>	Lock the per-Å scale to a fixed Å span and a square canvas (identical atom sizes across molecules)
<code>consistent_scale</code>	<code>auto</code>	Fixed per-Å scale when neither <code>width</code> nor <code>height</code> is given; <code>true/false</code> to force

DEVIATES FROM UPSTREAM

`tight_fit` and `padding: 10.0` are both port-side choices, not upstream defaults: upstream pads every side by the largest atom radius plus `padding: 20.0`. Set `tight_fit: false`, `padding: 20.0` for pixel-parity with upstream fixtures.

NOTE

`fixed_span` and `consistent_scale` solve the same underlying problem (keeping atoms the same visual size when rendering different molecules side by side) from two directions: `fixed_span` is explicit (you name the Å span); `consistent_scale` derives one automatically whenever you let the canvas auto-size (i.e. you passed neither `width` nor `height`).



`rotate: (x: 30deg, y: -45deg)`

13 Out of scope (for now)

Not every corner of upstream has made the trip over yet. Surfaces / molecular orbitals / cube files, stereochemistry labels, structural overlay & ensembles, convex-hull faces, crystal / periodic structures, the [skeletal](#) preset, and the remaining non-XYZ input formats (MOL/SDF/MOL2/SMILES/CIF) aren't ported. PDB is — see Section 2.2 — CIF is the natural next one, but upstream itself only supports it by calling out to a full external library ([ase](#)) rather than parsing it directly, so porting it faithfully is a larger undertaking than translating a file, and it's waiting on that being worth doing.

None of this is permanent: “out of scope” here means **not yet**, not **never**. If you hit one of these and it's blocking you, that's useful to know.

14 Citation

[xyzrender](#) is a Typst port: the chemistry (bond perception, aromaticity detection, presets, rendering approach) is xyzrender, written and maintained by Alister S. Goodfellow (github.com/aligfellow/xyzrender). This package exists to bring that work into Typst directly; none of the underlying chemistry or design originates here.

xyzrender itself builds on xyzgraph and graphRC for molecular graph construction — bond orders, aromaticity detection, and the rest of the cheminformatics this port’s own Rust/WASM plugin is a direct translation of. If you use [xyzrender-typst](#) (or upstream [xyzrender](#)) in published work, please cite the paper upstream asks to be cited for:

CITATION

A.S. Goodfellow* and B.N. Nguyen, *J. Chem. Theory Comput.*, 2026, DOI: 10.1021/acs.jctc.5c02073

```
@article{goodfellow2026xyzgraph,  
  author = {Goodfellow, A.S. and Nguyen, B.N.},  
  title = {Graph-Based Internal Coordinate Analysis for Transition State  
Characterization},  
  journal = {J. Chem. Theory Comput.},  
  year = {2026},  
  doi = {10.1021/acs.jctc.5c02073},  
}
```